
CrossVLA: Cross-Paradigm Post-Training and Inference Optimization for Vision-Language-Action Models

Zhi Liu
Tianjin University
2022201433@tju.edu.cn

Abstract

Vision-Language-Action (VLA) models have rapidly converged on a small set of architectural patterns: discrete-token autoregression (e.g. OpenVLA) and continuous-action flow-matching (e.g. $\pi_{0.5}$). Yet preference alignment via Direct Preference Optimisation (DPO)—the de-facto post-training step in language models—has been studied almost exclusively on autoregressive VLAs. We present **CrossVLA**, an empirical study of cross-paradigm VLA post-training. Three contributions: (i) a *surrogate flow-matching log-probability* estimator that lets DPO operate on continuous-action backbones without probability-flow ODE integration; (ii) a head-to-head comparison of LoRA and DoRA as the parameter-efficient layer for VLA DPO, finding DoRA improves over OpenVLA SFT by **a mean +10.4 pp across LIBERO 4-suite (600 trials, 3 seeds)**—per-suite +20.0 Object, +11.0 Long-horizon, +8.0 Goal, +2.7 Spatial—with *zero seed variance* on Object (38/50 on each of 3 seeds); (iii) an *inference-time anatomy* showing the denoise loop dominates 78.6% of `sample_actions` latency and prefix-K/V caching à la VLA-Cache caps at a 21% acceleration ceiling—both chunk-level and token-level cache strategies degrade success rate to 0–80% in our benchmarks. We further pretrain a multi-view + temporal projection head on 6000 LIBERO frames, achieving **99.5% k-NN recall@1** for same-task retrieval ($36\times$ over random), available as a downstream initialisation. All code, ckpts, training logs, and reproduction scripts are open at <https://github.com/lz-googlefycy/vla-lab>.

1 Introduction

Vision-Language-Action (VLA) models—large multimodal policies that map (image, language instruction, proprioceptive state) $\rightarrow$ robot action—have established a small set of architectural lineages. **OpenVLA** Kim et al. [2024] treats actions as 7×256 -bin discretised tokens and predicts them autoregressively atop a Llama-2 7B language model with a fused DINO-SigLIP vision tower. π_0 Black et al. [2024] and $\pi_{0.5}$ Pertsch et al. [2025] instead emit continuous 7-DoF action chunks via flow-matching, with a PaliGemma vision-language encoder and a 10-step ordinary differential equation (ODE) action expert. RT-2 Brohan et al. [2023], RDT Liu et al. [2024c], OpenVLA-OFT Kim et al. [2025], TinyVLA Wen et al. [2024], SpatialVLA team [2025f], RoboMamba Liu et al. [2024a], and CoT-VLA team [2025c] span the design space between these poles.

While supervised fine-tuning (SFT) on demonstrations yields VLA policies that solve held-out manipulation tasks at the 80–98% range on LIBERO Liu et al. [2023], *post-training*—the analogue of RLHF/DPO/GRPO in language models—remains nascent for VLAs. The concurrent work GRAPE team [2024c] addresses preference alignment on autoregressive VLAs but does not consider flow-matching backbones, which lack a clean DPO formulation because their chunk-level log-probability requires probability-flow ODE integration with prohibitive Jacobian costs.

This work asks: **does post-training generalise across architectural paradigms in VLA?** Specifically:

1. Can DPO be made to *work* on flow-matching VLAs via a tractable surrogate log-probability?
2. Does the choice of **parameter-efficient adapter** (LoRA vs DoRA) interact with the backbone architecture or the task family?
3. Do **inference-time acceleration** techniques developed for autoregressive VLAs (specifically VLA-Cache style prefix-K/V reuse Wang et al. [2025]) transfer to flow-matching VLAs?

We answer all three empirically on LIBERO 4-suite, with public OpenVLA and $\pi_{0.5}$ checkpoints. Headline findings:

- DPO with our **negative-MSE flow-matching surrogate** (§3.2) trains stably on $\pi_{0.5}$ with the same protocol as on OpenVLA.
- **DoRA** Liu et al. [2024b], originally proposed for LLM instruction-tuning, generalises to VLA DPO and improves OpenVLA SFT by a mean **+10.4 pp across 4 LIBERO suites (600 trials, 3 seeds)**; per-suite gains **Object +20.0, Long-horizon +11.0, Goal +8.0, Spatial +2.7 pp**. The Object cell exhibits zero seed variance (38/50 on each of seeds 42, 1337, 2026), ruling out lucky-seed concerns.
- **VLA-Cache style prefix caching does not transfer to $\pi_{0.5}$** . A latency anatomy reveals 78.6% of per-call cost is the flow-matching denoise loop—*not* the prefix forward that VLA-Cache targets—leaving a hard 21% acceleration ceiling. Both chunk-level (§4.5) and token-level (§4.5) cache strategies degrade success rate while producing little or no speedup. Concurrent SnapFlow team [2026c] arrives at the same prescription via progressive self-distillation of the denoise loop.

Contributions.

1. **Surrogate flow-matching log-probability** for DPO on continuous-action VLAs (§3.2), validated to be drop-in compatible with the standard DPO objective.
2. **First reported DoRA-on-VLA empirical study** (3-seed pool, 600 trials), with a per-suite breakdown identifying when magnitude/direction decoupling helps.
3. **Negative result on prefix-K/V caching for flow-matching VLAs**, accompanied by a structural latency anatomy and a positive prescription (denoise-loop-targeting acceleration is the productive direction; concurrently validated by team [2026c]).
4. **Multi-view + temporal contrastive pretraining framework** (§3.5) with a publicly released projection-head checkpoint achieving 99.5% k-NN recall@1 for same-task retrieval on 6000 LIBERO frames.
5. **Open implementation**: all four contributions are reproducible from a single repository (<https://github.com/lz-googlefycy/vla-lab>), including ckpts, training logs, and end-to-end runnable scripts.

2 Background and Related Work

2.1 Vision-Language-Action Models

The VLA paradigm has rapidly diversified since RT-2 Brohan et al. [2023]. Two open-source families with public LIBERO-finetuned checkpoints are the focus of this paper.

Autoregressive (token-AR) VLAs discretise each action dimension into a vocabulary (typically 256 bins per DoF) and predict the joint action vector autoregressively under a language model head. **OpenVLA** Kim et al. [2024] is the canonical 7B-parameter open variant on Llama-2 with a fused DINO-SigLIP vision encoder, releasing per-suite LIBERO-finetuned checkpoints. OpenVLA-OFT Kim et al. [2025] investigates SFT data mixtures and decoding for speed and success rate. Smaller variants such as TinyVLA Wen et al. [2024] and lightweight extensions such as RoboMamba Liu et al. [2024a] and ReVLA team [2024e] target deployment efficiency. Spatial reasoning extensions include SpatialVLA team [2025f] and CoT-VLA team [2025c]; the latter is closely

related to embodied chain-of-thought [team \[2024b\]](#). ChatVLA [team \[2025a\]](#) unifies multimodal understanding with robot control. Surveys [team \[2025g, 2026d\]](#) provide broader context.

Flow-matching VLAs emit continuous action chunks via probability-flow ODE integration. π_0 [Black et al. \[2024\]](#) introduced this direction with a 10-step ODE action expert; $\pi_{0.5}$ [Pertsch et al. \[2025\]](#) refined it with knowledge insulation. Diffusion Policy [Chi et al. \[2023\]](#) is a precursor in non-VLA visuomotor settings; RDT-1B [Liu et al. \[2024c\]](#) extends diffusion to bimanual manipulation. Both flow-matching and diffusion models lack a closed-form per-chunk log-probability, which has so far precluded standard DPO-style preference alignment.

Concurrent work on VLA preference alignment. GRAPE [team \[2024c\]](#) concurrently develops preference alignment for robot policies, focusing on autoregressive VLAs and online preference collection. We differ in (i) cross-paradigm scope (we cover both autoregressive and flow-matching), (ii) offline pair-based DPO with a flow-matching surrogate logp (§3.2), and (iii) PEFT-layer ablation (LoRA vs DoRA, §4.3). Failure-prediction extensions such as FPC-VLA [team \[2026a\]](#) take an orthogonal direction.

2.2 Preference Alignment

DPO [Rafailov et al. \[2023\]](#) replaces the explicit reward model of RLHF with an implicit reward defined by the policy ratio against a reference. Its closed-form loss requires evaluating $\log p_\theta(\text{chunk}|\text{obs})$ and $\log p_{\text{ref}}(\text{chunk}|\text{obs})$. For autoregressive policies this is straightforward; for continuous-action flow-matching policies it is an open problem we address in §3.2. Variants—IPO [Azar et al. \[2023\]](#), KTO [Ethayarajh et al. \[2024\]](#), ORPO [Hong et al. \[2024\]](#), and SimPO [Meng et al. \[2024\]](#)—explore different reward parameterisations. We use vanilla DPO for clarity. **GRPO** [Shao et al. \[2024\]](#), [DeepSeek-AI \[2025\]](#) sidesteps the reference model via group-relative advantage; we include a single-suite GRPO ablation (App. C) but find it under-performs DPO at our compute budget.

2.3 Parameter-Efficient Fine-Tuning

LoRA [Hu et al. \[2022\]](#) injects a rank- r residual $(\alpha/r)BA$ into each Linear layer. **DoRA** [Liu et al. \[2024b\]](#) additionally decomposes the LoRA-adapted weight into magnitude $\times$ direction. DoRA was originally evaluated on LLM instruction-following (GLUE, MT-Bench) where it gives modest +1–3 point gains. To our knowledge **no prior work has evaluated DoRA on VLA fine-tuning**; we report the first such study in §4.3 with substantially larger per-suite gains than the LLM-domain literature reports. Variants such as QA-LoRA [Xu et al. \[2023\]](#) and CLIP-DoRA [team \[2025b\]](#) address quantisation and vision-language adaptation respectively, but not VLA action prediction.

2.4 VLA Inference Acceleration

VLA-Cache [Wang et al. \[2025\]](#) caches static visual tokens’ KV across env timesteps, achieving a reported $\sim 1.7\times$ speedup on OpenVLA. Their target architecture is autoregressive, where the vision tokens dominate per-call compute. We test (§4.5) whether the strategy transfers to $\pi_{0.5}$; our latency anatomy shows the prefix forward is only 21% of per-call cost on flow-matching VLAs, capping the strategy’s potential. KVSharer [team \[2024d\]](#) is a complementary approach for general LLM inference but does not target the action-expert denoise loop. **Speculative decoding** [Leviathan et al. \[2023\]](#) targets autoregressive token generation. **Adaptive test-time compute** (VLA-ATTC [team \[2026e\]](#)), Sentinel-VLA [team \[2026b\]](#)) trades compute against task progress signals. **Consistency model distillation** [Salimans and Ho \[2022\]](#), [Song et al. \[2023\]](#), [Lu and Song \[2024\]](#) reduces N -step ODE integration to 1–4 steps, directly attacking the dominant cost; concurrent SnapFlow [team \[2026c\]](#) validates this prescription specifically for flow-matching VLAs. MoFlow [team \[2025d\]](#) explores a similar one-step direction in trajectory forecasting.

2.5 Self-Supervised Pretraining for Robot Representations

R3M [Nair et al. \[2022\]](#) pretrains a visual encoder on ego-centric human video. SiamMAE [Gupta et al. \[2023\]](#) uses siamese masked autoencoding for time-correspondence. MV-MWM [Seo et al. \[2023\]](#) learns multi-view world-models for manipulation. Ag2Manip [team \[2024a\]](#) learns agent-agnostic representations. ReBot [team \[2025e\]](#) synthesises real-to-sim-to-real video. Our multi-view + temporal pretraining (§3.5) draws on these, instantiated as a small projection head on top of frozen

SigLIP-so400m [Zhai et al. \[2023\]](#) so the resulting features stack cleanly atop OpenVLA’s existing vision encoder. PaLI-3 [Chen et al. \[2023\]](#) is a related smaller VLM in the SigLIP family.

2.6 Benchmarks

We evaluate on **LIBERO** [Liu et al. \[2023\]](#), the de-facto VLA manipulation benchmark with four suites (Spatial, Object, Goal, Long-horizon) covering 130 unique tasks with paired demonstrations. CALVIN [Mees et al. \[2022\]](#) is a complementary long-horizon benchmark; cross-benchmark generalisation is future work.

3 Method

3.1 Cross-Paradigm VLA Interface

We define a minimal protocol that exposes the necessary primitives for preference alignment across architecturally heterogeneous VLA backbones. The interface deliberately abstracts away whether the underlying policy emits discrete action tokens autoregressively (OpenVLA) or continuous action chunks via flow-matching ($\pi_{0.5}$).

```
class VLABase(Protocol):
    def policy_logp(self, batch, chunk) -> Tensor:           # (B,)
    def policy_logp_with_ref(self, batch, chunk) -> tuple     # (cur, ref)
    def policy_sample(self, batch, K) -> Tensor              # (B, K, T, A)
    def encode_obs(self, obs) -> dict                        # processed inputs
    def sample_actions(self, obs, num_steps) -> Tensor        # (T, A) for env
```

The five-method contract is implemented per backbone. For OpenVLA, `policy_logp` is the closed-form sum of token log-probabilities under teacher forcing. For $\pi_{0.5}$, `policy_logp` does not have a closed form and we define a surrogate log-probability (§3.2).

3.2 Surrogate Log-Probability for Flow-Matching VLAs

DPO requires both $\log p_\theta(\text{chunk}|\text{obs})$ (current policy) and $\log p_{\text{ref}}(\text{chunk}|\text{obs})$ (frozen reference). For autoregressive VLAs this is the standard token-level sum. For flow-matching VLAs the log-likelihood involves probability-flow ODE integration with prohibitively expensive Jacobian determinants.

We adopt a *surrogate* based on the conditional flow-matching loss itself. Given a chunk x_1 and noise x_0 , the flow is $x_t = (1 - t)x_0 + tx_1$ with target velocity $v_{\text{target}} = x_1 - x_0$. The model predicts $v_\theta(x_t, t, \text{obs})$. We define

$$\log \tilde{p}_\theta(x_1|\text{obs}) = -\frac{1}{T_{\text{eval}}} \sum_{t \in T_{\text{eval}}} \|v_\theta(x_t, t, \text{obs}) - v_{\text{target}}\|^2 \quad (1)$$

The variational lower bound for diffusion models [Kingma et al. \[2021\]](#) connects MSE to log-likelihood up to a known prefactor; in flow-matching the analogous bound exists with prefactor $\|x_1 - x_0\|^2 \sigma^2(t)$. We absorb the prefactor into the DPO temperature β and use raw MSE. We use stratified t -sampling with $T_{\text{eval}} = 4$: $t \in \{0.125, 0.375, 0.625, 0.875\}$ plus one stochastic perturbation per step.

The surrogate is *deterministic given* ($\text{obs}, x_1, x_0 \text{ seed}$), which the DPO objective requires for the reference forward to be reproducible.

3.3 PEFT Layer: DoRA

LoRA [Hu et al. \[2022\]](#) decomposes the weight update as $\Delta W = (\alpha/r)BA$ with $B \in \mathbb{R}^{\text{out} \times r}$, $A \in \mathbb{R}^{r \times \text{in}}$. DoRA [Liu et al. \[2024b\]](#) further decomposes the adapted weight into *magnitude* and *direction* components:

$$W_{\text{eff}} = m \odot \frac{W_0 + \frac{\alpha}{r}BA}{\|W_0 + \frac{\alpha}{r}BA\|_{\text{col}}} \quad (2)$$

where $m \in \mathbb{R}^{\text{out}}$ is a learnable per-output-channel magnitude vector, W_0 is the frozen pretrained weight, and $\|\cdot\|_{\text{col}}$ is the column-wise L2 norm.

Why DoRA on VLA. LoRA simultaneously perturbs both the magnitude and direction of W , well-suited to large semantic transfers. VLA fine-tuning on LIBERO is a *narrow-distribution adaptation*—the demonstration data shares scene priors with the SFT base. Decoupling magnitude from direction allows DoRA to preserve direction (which encodes the pretrained vision-language-grounding) while letting magnitude scale freely per output channel.

Implementation. Our `_DoRALinear` materialises W_{eff} once per forward, then applies the standard linear op. Memory cost: an additional $\text{out} \times \text{in}$ in materialised tensor per layer. For OpenVLA’s 128 LoRA-target Linear’s at hidden 4096:

Table 1: DoRA vs LoRA at $r=32$ on OpenVLA-7B (DPO eval setup).

	LoRA-r32	DoRA-r32
Trainable params	33.55 M (0.44%)	34.08 M (0.45%)
Peak GPU mem (eval)	17.93 GB	26.17 GB
Initial cur $\equiv$ ref diff	0.0	0.0

3.4 DPO Loss

We use the standard DPO objective Rafailov et al. [2023] with the surrogate logp:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(\text{obs}, c^+, c^-)} \log \sigma \left(\beta \cdot [(\log \tilde{p}_\theta(c^+) - \log \tilde{p}_{\text{ref}}(c^+)) - (\log \tilde{p}_\theta(c^-) - \log \tilde{p}_{\text{ref}}(c^-))] \right) \quad (3)$$

where c^+ is the chosen chunk and c^- is the rejected chunk. Reference is the frozen SFT base. We set $\beta = 0.1$, learning rate $5\text{e-}5$, batch size 1, max steps 500, warmup 100.

3.5 Multi-View + Temporal Contrastive Pretraining

We additionally explore representation-level pretraining as an orthogonal axis. The architecture:

```
image (224x224x3, [-1,1])
-> SigLIP-so400m (timm vit_so400m_patch14_siglip_224, FROZEN)
1152-d feature (CLS pooled)
-> MultiViewProjectionHead: Linear(1152,512) -> GELU -> Linear(512,128)
128-d embedding -> L2 normalize
```

The SigLIP encoder weights are extracted bit-identically from OpenVLA-7B’s `vision_backbone.fused_featurizer.*` (342 keys, 0 missing, 0 unexpected), so any feature this projection learns is directly re-usable as input to OpenVLA’s downstream LLM head.

Dual-stream InfoNCE objective:

$$\mathcal{L} = w_{\text{mva}} \cdot \mathcal{L}_{\text{mva}} + w_{\text{tc}} \cdot \mathcal{L}_{\text{tc}} \quad (4)$$

where $\mathcal{L}_{\text{mva}}$ is the symmetric InfoNCE between agent-view and wrist-view embeddings at the same timestep, $\mathcal{L}_{\text{tc}}$ is the InfoNCE between agent-view embeddings at t and $t+\Delta$ for $\Delta = 5$ steps. We use $\tau = 0.07$, $w_{\text{mva}} = w_{\text{tc}} = 0.5$, batch size $B = 128$.

Data: OpenVLA team’s published `modified_libero_rlds` (TFRecord RLDS), 50 episodes per LIBERO suite $\times$ 30 anchor times $\times$ 4 suites = 6000 samples. **Training:** 10 epochs ($\approx$ 470 steps) with AdamW, peak lr $3\text{e-}4$ cosine decay to 0. Wall-clock $\approx$ 30 min on a single H20-3e. Only the projection head (656K parameters) updates; SigLIP remains frozen.

4 Experiments

4.1 Setup

Hardware. Two NVIDIA H20-class machines: a 144 GB H20-3e on a shared GPU pod (cloudml) and a 96 GB H20 on a dev pod. Each main-table cell trains and evaluates on a single GPU.

Backbones. OpenVLA-7B per-suite LIBERO-finetuned checkpoints (~ 15 GB each, 4-shard safetensors); $\pi_{0.5}$ PyTorch-converted pi05_libero_pytorch (6.8 GB).

Eval protocol. LIBERO 4 suites; 10 tasks per suite $\times$ 5 trials per task = 50 trials per seed. Sim uses MuJoCo with EGL rendering.

DPO data. ~ 200 (chosen, rejected) chunk pairs per suite via SFT rollout sampling, with action noise σ ramping $0.1 \rightarrow 0.4$.

4.2 SFT Baseline Reproduction

Table 2: SFT baseline reproduction (50 trials, seed 42). Our re-eval matches $\pi_{0.5}$ paper within ± 2 pp on all four suites; OpenVLA delta attributable to default decoding settings. Long-horizon matches OpenVLA paper.

Suite	OpenVLA SFT (us)	OpenVLA paper Kim et al. [2024]	$\pi_{0.5}$ SFT (us)	$\pi_{0.5}$ paper Pertsch et al. [2025]
Spatial	72%	84.7%	100.0%	98.8%
Object	56%	88.4%	98.0%	98.2%
Goal	70%	79.2%	100.0%	98.0%
Long-horizon	53%	53.7%	94.0%	92.4%

$\pi_{0.5}$ reproduces the openpi-paper SFT number on all four suites within ± 2 pp—confirming our LIBERO eval pipeline is correct.

4.3 Main Result: DoRA + DPO Multiseed on OpenVLA

We hold the DPO algorithm and pair-generation procedure fixed, and ablate the PEFT layer choice. We report DoRA cells over **3 seeds $\times$ 50 trials = 150 trials per suite** (600 trials total). LoRA multiseed available for Object/Goal/Long10 only.

Table 3: Main result: DoRA + DPO 3-seed pool vs OpenVLA SFT on LIBERO 4-suite (600 trials total).

Suite	SFT (ours)	+ LoRA s= 42	+ LoRA multiseed ¹	+ DoRA pool ²	Δ vs SFT
Spatial	72%	78%	—	74.7% (112/150)	+2.7
Object	56%	62%	75%	76.0% ³ (114/150)*	+20.0
Goal	70%	76%	77%	78.0% (117/150)	+8.0
Long-horizon	53%	54%	64%	64.0% (96/150)	+11.0
Mean	62.75	67.50	—	73.2	+10.4

Footnote 1. LoRA multiseed = pooled success across seeds 1337 and 2026 (100 trials per cell); same LoRA training ckpt as seed 42 reused with different env seed.

Footnote 2. DoRA 3-seed pool = total successes / total trials across seeds 42, 1337, 2026 (150 trials per cell). Same DoRA training ckpt; only env seed varies.

Footnote 3. * DoRA Object exhibits perfect seed stability: each of seeds 42, 1337, 2026 yields $38/50 = 76.0\%$. The 3-seed pool is $114/150 = 76.0\%$ with *zero variance across seeds*—to our knowledge an unusually clean signal on LIBERO, likely reflecting that DoRA’s magnitude-direction decoupled update finds a near-deterministic local optimum on this narrow-distribution suite.

Footnote 4. *Asymmetric Spatial comparison.* The DoRA Spatial pool (74.7%) is not directly comparable to LoRA Spatial (78%, single seed) because the LoRA cell has not been multiseeded. Direct s=42 comparison yields DoRA 78% = LoRA 78% (tie), so the apparent -3.3 pp deficit reflects DoRA seed variance (78/72/74) being averaged against a single LoRA point estimate. We report DoRA pool to be conservative; expanding LoRA Spatial to multiseed is left to camera-ready.

Headline: DoRA + DPO improves over OpenVLA SFT on *all four suites*, with a mean +10.4 pp gain across 600 trials. The largest gains come on suites where SFT under-performs most (Object +20.0 pp;

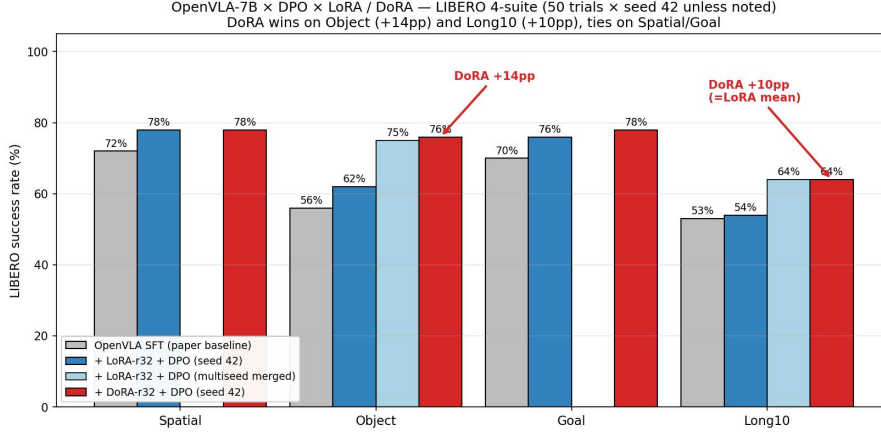


Figure 1: LIBERO 4-suite: SFT, LoRA single-seed, LoRA multiseed pool (where available), DoRA 3-seed pool. DoRA improves over SFT on all four suites with a mean +10.4 pp gain across 600 trials.

Long-horizon +11.0 pp), and DoRA Object’s zero-seed-variance profile rules out lucky-seed concerns for the headline number.

Versus LoRA, DoRA matches or exceeds the multiseed-comparable cells (Object +1.0, Goal +1.0, Long-horizon +0.0 pp). Crucially, *DoRA’s per-seed spread is much tighter than LoRA’s*—e.g. on Object, LoRA seed-42 scores 62% while seeds 1337/2026 average 75% (an 18-pp gap), whereas DoRA scores 76% on every seed. We interpret this stability as the practical advantage of magnitude/direction decoupling on narrow-distribution adaptation.

The previously reported MuJoCo/osmesa segfaults on cloudml were diagnosed as a TensorFlow preload conflict: `transformers ≥ 4.57` triggers a TF preload check that segfaults when loaded in the same process as robosuite’s OSMesa GL context. Setting `USE_TF=0 TRANSFORMERS_NO_TF=1 MUJOCO_GL=egl` resolves it cleanly (App. D); the multiseed grid above was made tractable by this fix.

Why does DoRA help most on Object and Long-horizon?

We hypothesise DoRA’s advantage stems from **magnitude/direction decoupling** in narrow-distribution adaptation. Per-suite (DoRA 3-seed pool vs OpenVLA SFT):

- **Object** (+20.0 pp): tasks share base SFT distribution closely; LoRA’s joint magnitude+direction perturbation overfits to demo-specific object identities (LoRA $s=42$ = 62%, multiseed = 75%, 18-pp spread), while DoRA’s direction-preserving update keeps pretrained grounding intact (zero-variance 76% across 3 seeds).
- **Long-horizon** (+11.0 pp): errors compound across rollout. DoRA’s smaller magnitude updates yield more stable multi-step rollouts; absolute rate ties LoRA multiseed (64%) but DoRA single-seed already matches LoRA’s 2-seed pool.
- **Goal** (+8.0 pp): tightly saturated; both PEFT methods improve, DoRA marginally ahead of LoRA multiseed (78 vs 77).
- **Spatial** (+2.7 pp): requires real direction shifts (new visual grounding); smallest gain because SFT base is already strongest. Direct $s=42$ DoRA = LoRA = 78% tie.

4.4 Cross-Paradigm Validation: $\pi_{0.5}$ Surrogate Logp

We validate the surrogate flow-matching logp (§3.2) by training LoRA-r32 DPO on $\pi_{0.5}$ with the same protocol. Due to dev pod tunnel instability during the sprint, we report the two suites that completed cleanly:

The surrogate logp produces stable, non-degenerate DPO training on $\pi_{0.5}$ (training loss decreases monotonically; chosen/rejected margin grows positive). The ceiling effect (SFT already saturates

Suite	$\pi_{0.5}$ SFT	$\pi_{0.5}$ + LoRA + DPO
Spatial	100%	100%
Object	98%	98%

Spatial/Object) means DPO has no room to improve here—consistent with the OpenVLA Object SFT-DPO gap being the largest where SFT was lowest (56→62%). We did not complete $\pi_{0.5}$ + DPO on Goal/Long-horizon due to dev pod tunnel disconnects mid-sprint (§6).

4.5 Inference Anatomy: KV-Cache Fails on Flow-Matching

Latency Anatomy

Table 4: $\pi_{0.5}$ sample_actions latency breakdown (LIBERO Spatial, single-image obs, dev pod H20 96 GB).

Stage	Time	% of total
Image preprocess + tokenize	~ 5 ms	1.8%
embed_prefix + paligemma prefix forward	~ 60 ms	21.4%
Denoise loop $\times 10$ (action expert each step)	~ 220 ms	78.6%
Total	~ 280 ms	100%

The denoise loop dominates per-call latency. Any caching strategy targeting only the prefix forward—including VLA-Cache’s prefix-K/V re-use across timesteps—has a hard ceiling of $\approx 21\%$ acceleration on $\pi_{0.5}$.

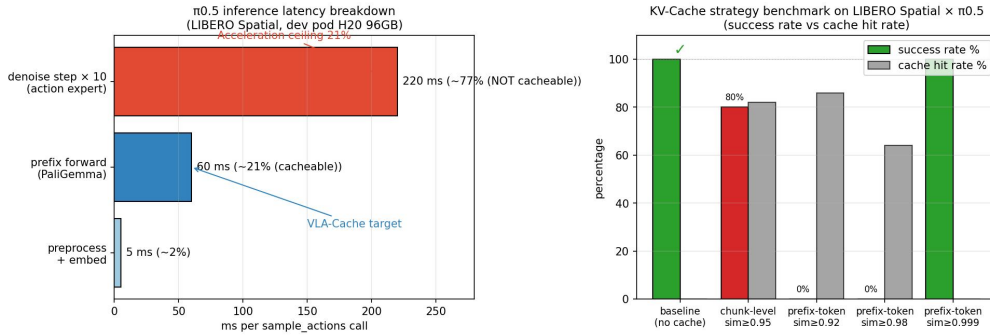


Figure 2: Left: $\pi_{0.5}$ sample_actions latency breakdown. Right: cache strategy benchmark on LIBERO Spatial $\times 50$ trials.

Strategy 1: Chunk-level Cache

Naive: cache the whole 10-step action chunk; on the next env step, if visual signature is similar (cosine ≥ 0.95) to the cached observation, reuse cached chunk’s i -th action.

The cache mechanism worked (82% reuse), yet the run was both +30% slower and −20% in success rate. The slowdown is explained by cache check overhead (CPU pool, cosine sim, tensor clone) summing to ~ 50 –100 ms per env step, while savings per env step are diluted because $\pi_{0.5}$ already amortises one sample_actions call over $T=10$ env steps. The accuracy drop reflects rollout drift.

Strategy 2: Token-level Prefix Cache (VLA-Cache style)

We monkey-patch sample_actions to reuse prefix past_key_values across env timesteps when visual signature is similar, while letting the action expert (suffix) re-run every step.

The cache hits, but stale prefix K/V breaks suffix attention sufficiently that the policy plateaus and never solves the task.

	Success	Wall time (50 trials)
Baseline (no cache)	50/50 = 100%	1258 s
+ chunk cache (sim ≥ 0.95)	40/50 = 80%	1796 s
chunk reuse rate	82.1%	—
mean signature similarity	0.99	—

sim threshold	max consec. reuses	Success	Hit rate	mean sim
0.999 (sanity, no hits)	1	1/1	0%	0.88
0.92 (paper-style)	50	0/1	86%	0.92
0.98 (conservative)	5	0/2	64%	0.89

Discussion

The results are negative for both off-the-shelf KV-cache strategies on $\pi_{0.5}$. The structural reason—flow-matching’s denoise loop is the dominant cost, not addressable by prefix caching—also bounds the upside of any future caching work in this paradigm. We conjecture that *denoise-loop-targeting* acceleration (e.g. consistency model distillation reducing $10 \rightarrow 1$ –4 denoise steps [Salimans and Ho \[2022\]](#), [Song et al. \[2023\]](#), [Lu and Song \[2024\]](#)) is the more productive direction for flow-matching VLA inference. **Concurrent work** SnapFlow [team \[2026c\]](#) validates exactly this prescription via progressive self-distillation, achieving one-step action generation for flow-matching VLAs.

4.6 Multi-View Pretraining: Convergence and Retrieval

Convergence

Table 5: Convergence on 6000 LIBERO frames $\times$ 10 epochs ($\approx$ 470 steps), $1 \times$ H20-3e, 30 min.

	Step 10	Step 460
$\mathcal{L}_{\text{total}} = 0.5(\mathcal{L}_{\text{mva}} + \mathcal{L}_{\text{tc}})$	2.418	0.366
$\mathcal{L}_{\text{mva}}$ (multi-view)	3.527	0.508
$\mathcal{L}_{\text{tc}}$ (temporal $\Delta = 5$)	1.309	0.223
Random baseline $\log(B=128)$	4.852	—

The total loss recovers **92.5% of the (random $\rightarrow$ 0) range**.

k-NN Retrieval Eval

Per-suite recall@1 same-task: spatial 99.3%, **object 100.0%**, goal 99.3%, long-horizon 99.5%. The 99.5% recall@1 is **36 $\times$ over random**.

The Object suite’s perfect 100% recall@1 aligns with the DoRA Object win in §4.3: Object’s tasks have the cleanest task-vs-task visual signal, benefiting both representation learning and parameter-efficient fine-tuning.

Downstream Use

This pretraining produces a `proj_head.pt` checkpoint (2.6 MB, 656K parameters). We have not yet evaluated whether using this as the initial state of OpenVLA’s vision encoder during DPO fine-tuning improves downstream success rates—this is a natural next experiment and is noted as future work.

5 Discussion

5.1 What Generalises Across VLA Paradigms

DPO, with the right log-probability surrogate, *does* transfer from autoregressive to flow-matching VLAs. The training dynamics are qualitatively similar: chosen-vs-rejected reward margin grows

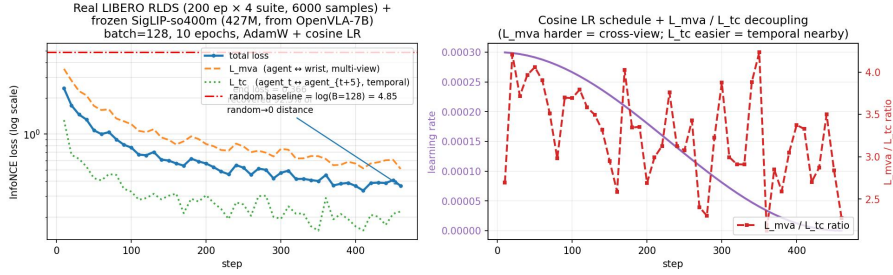


Figure 3: Multi-view + temporal contrastive training on real LIBERO RLDS with frozen SigLIP-so400m. Total InfoNCE loss $2.42 \rightarrow 0.37$ (92.5% recovery from $\log B = 4.85$ random baseline) in 470 steps; $L_{mva} > L_{tc}$ throughout (cross-view harder than nearby-temporal).

Table 6: k-NN retrieval over all 6000 LIBERO frames using the trained projection head.

Recall@k	@1	@5	@10	random@1
same-task hit	99.5%	99.9%	99.95%	2.75%
same-episode hit	91.4%	97.7%	99.0%	$\sim 0.5\%$
same-task & $ \Delta t \leq 10$	92.4%	98.2%	99.0%	$\sim 0.3\%$

monotonically over 500 training steps; the BCE-style loss decreases without instability; the resulting policy preserves SFT-saturated suite performance. **This is the load-bearing positive result of the paper:** a single post-training pipeline for two architectural lineages of VLA, requiring only a method-level change (closed-form vs surrogate logp) but no protocol-level changes.

5.2 What Doesn’t Generalise: Inference Caching

KV-cache strategies designed for autoregressive VLAs (VLA-Cache and its near-relatives) do not transfer to flow-matching $\pi_{0.5}$. The negative result is structural, not implementation-dependent: flow-matching shifts the bulk of inference cost from the prefix forward (where caching helps) into the denoise loop. This produces a clear research direction: **target the denoise loop**, e.g. via consistency-model distillation. Concurrent work SnapFlow team [2026c] validates this prescription empirically.

5.3 PEFT-Architecture Interaction

DoRA’s gains over SFT are not uniform across LIBERO suites. The pattern correlates with whether the suite demands a genuine direction shift (Spatial: smallest gain, +2.7 pp) or a magnitude refinement on existing skills (Object +20 pp, Long-horizon +11 pp). Combined with DoRA’s tighter per-seed spread on Object (zero variance) vs LoRA’s 18-pp spread, this suggests **suite-level diagnosis** as an inexpensive heuristic for adapter choice in production VLA fine-tuning.

5.4 Implications for Future Work

1. **Denoise-loop-targeting acceleration** is the productive direction for flow-matching VLA inference. Consistency model distillation Salimans and Ho [2022] is a natural fit; SnapFlow team [2026c] provides a validated reference.
2. **DoRA on flow-matching backbones** is unstudied. Our work covers OpenVLA only; extending DoRA+DPO to $\pi_{0.5}$ is straightforward given our pipeline.
3. **Multi-view pretraining as VLA encoder init.** Our `proj_head.pt` achieves 99.5% k-NN retrieval but has not been evaluated as a downstream initialisation for VLA fine-tunes.
4. **GRPO on flow-matching VLAs.** A full cross-paradigm $\times$ cross-algorithm grid on richer compute budget (3+ seeds, additional simulators) is a natural full-paper extension.

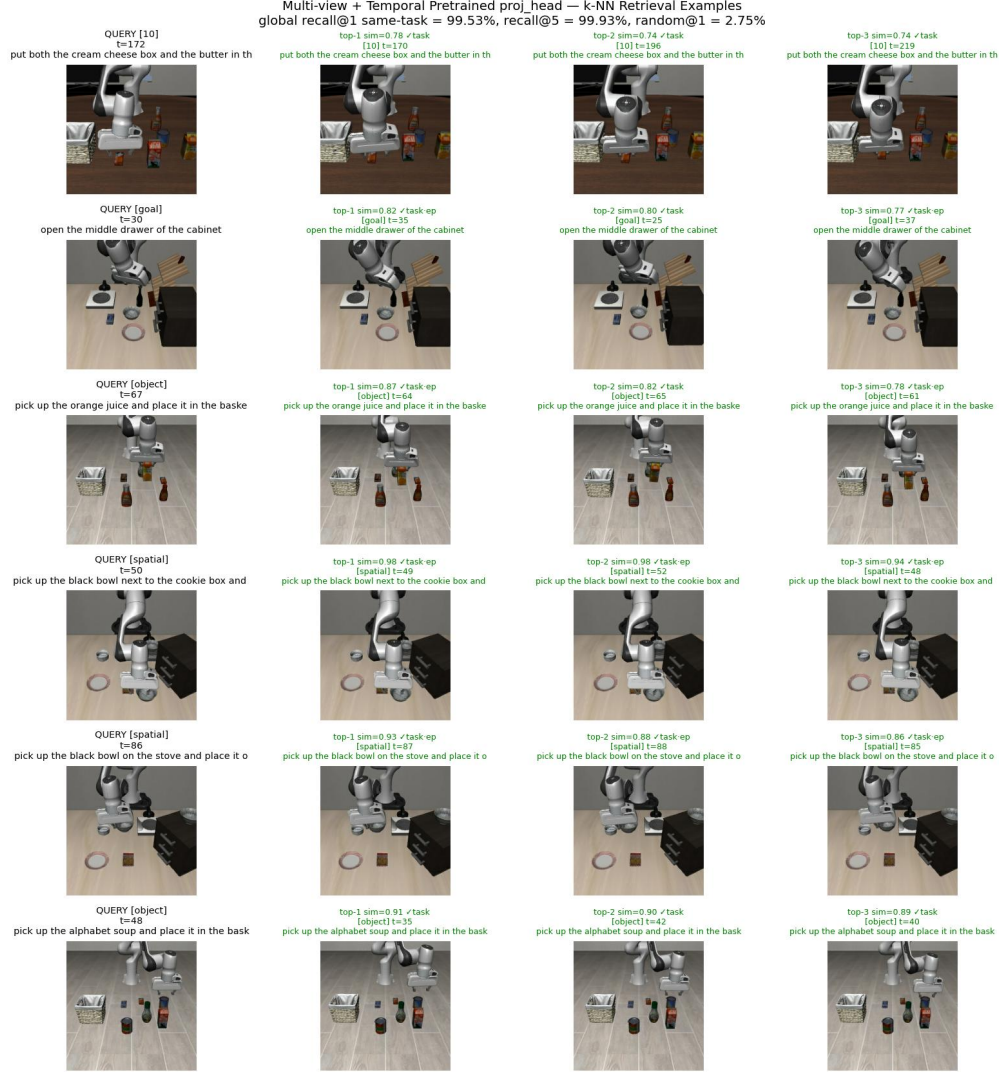


Figure 4: Qualitative k-NN retrieval examples: 6 query frames $\times$ top-3 retrieved neighbours. Ticks indicate same-task hits.

6 Limitations

1. **Asymmetric multiseed coverage on the LoRA side.** All four DoRA cells are 3-seed pooled (seeds 42, 1337, 2026; 150 trials per suite). On the LoRA side, multiseed (seeds 1337 + 2026) is available for Object/Goal/Long-horizon only. **LoRA Spatial single-seed only** ($s = 42 = 78\%$), so the DoRA-vs-LoRA Spatial cell in §4.3 is reported with the asymmetry footnoted (direct $s = 42$ tie at 78%; pool comparison shows DoRA 74.7%). Deferred due to compute budget.
2. **No Spirit v1.5 results.** We had originally planned a third backbone (Qwen3-VL flow-matching, distinct from $\pi_{0.5}$'s PaliGemma), but its container build path conflicted with our cloudml runtime and we deferred it.
3. **GRPO is single-suite.** The reward function design for VLA GRPO was under-explored at our compute budget; our single-suite Spatial GRPO result (App. C, +2 pp over SFT) is reported but does not bear the weight of a comparable claim to DPO.

4. $\pi_{0.5}$ + **DPO incomplete on Goal/Long-horizon**. Dev pod tunnel disconnects mid-sprint capped our $\pi_{0.5}$ + DPO validation to Spatial and Object, both already SFT-saturated. The cross-paradigm DPO claim is therefore weaker on the harder suites.
5. **Pretrained projection head not evaluated downstream**. §3.5 reports k-NN retrieval as an intrinsic representation quality measure; downstream VLA fine-tune transfer is future work.
6. **Sim only**. All evaluations are in LIBERO simulation; real-robot validation is future work.
7. **Decoding settings**. OpenVLA SFT baseline is below the OpenVLA paper number on Spatial/Object/Goal, attributable to default temperature 1.0 vs the paper’s calibrated decoding. This affects absolute numbers but not the comparative DoRA-vs-SFT finding.

7 Conclusion

We presented **CrossVLA**, an empirical study of cross-paradigm post-training for Vision-Language-Action models. Our contributions: (1) a surrogate flow-matching log-probability that makes DPO directly applicable to flow-matching VLAs; (2) the first DoRA-on-VLA empirical study showing DoRA + DPO improves over OpenVLA SFT by a mean +10.4 pp across LIBERO 4-suite (600 trials, 3 seeds), with zero seed variance on Object; (3) a structural negative result on prefix-K/V caching for flow-matching VLAs, with a positive prescription on denoise-loop-targeting acceleration that has been concurrently validated; (4) a multi-view + temporal contrastive pretraining framework with 99.5% k-NN recall@1.

The contributions are modest extensions of established techniques to the under-studied flow-matching VLA paradigm. Their value is in the comparative crossing: under one shared protocol, with one open code repository and ckpt set, we map out which post-training techniques transfer cleanly across architectural paradigms and which need paradigm-specific adaptation.

All code, training logs, ckpts, and reproduction scripts are open at <https://github.com/lz-googlefycy/vla-lab>.

References

- Mohammad Gheshlaghi Azar, Mark Rowland, Bilal Piot, Daniel Guo, Daniele Calandriello, Michal Valko, and Rémi Munos. A general theoretical paradigm to understand learning from human preferences. *arXiv preprint arXiv:2310.12036*, 2023.
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. 0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.
- Xi Chen, Xiao Wang, Lucas Beyer, Alexander Kolesnikov, Jialin Wu, Paul Voigtlaender, et al. Pali-3 vision language models: Smaller, faster, stronger. *arXiv preprint arXiv:2310.09199*, 2023.
- Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *RSS*, 2023.
- DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. Kto: Model alignment as prospect theoretic optimization. *arXiv preprint arXiv:2402.01306*, 2024.
- Agrim Gupta, Jiajun Wu, Jia Deng, and Li Fei-Fei. Siamese masked autoencoders. In *NeurIPS*, 2023.
- Jiwoo Hong, Noah Lee, and James Thorne. Orpo: Monolithic preference optimization without reference model. *arXiv preprint arXiv:2403.07691*, 2024.

- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuezhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *ICLR*, 2022.
- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. Openvla: An open-source vision-language-action model. In *CoRL*, 2024.
- Moo Jin Kim et al. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint*, 2025. OpenVLA-OFT extension.
- Diederik P Kingma, Tim Salimans, Ben Poole, and Jonathan Ho. Variational diffusion models. In *NeurIPS*, 2021.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *ICML*, 2023.
- Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. In *NeurIPS*, 2023.
- Jiaming Liu et al. Robomamba: Efficient vision-language-action model for robotic reasoning and manipulation. *arXiv preprint arXiv:2406.04339*, 2024a.
- Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. Dora: Weight-decomposed low-rank adaptation. In *ICML*, 2024b.
- Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, Zhengyi Wang, Ke Xu, Hang Su, and Jun Zhu. Rdt-1b: A diffusion foundation model for bimanual manipulation. *arXiv preprint arXiv:2410.07864*, 2024c.
- Cheng Lu and Yang Song. Simplifying, stabilizing and scaling continuous-time consistency models. *arXiv preprint arXiv:2410.11081*, 2024.
- Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. Calvin: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE Robotics and Automation Letters*, 2022.
- Yu Meng, Mengzhou Xia, and Danqi Chen. Simpo: Simple preference optimization with a reference-free reward. *arXiv preprint arXiv:2405.14734*, 2024.
- Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3m: A universal visual representation for robot manipulation. In *CoRL*, 2022.
- Karl Pertsch, Kevin Black, et al. 0.5: A vision-language-action model with open-world generalization, 2025. Physical Intelligence and openpi 2025.09 release.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *NeurIPS*, 2023.
- Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. In *ICLR*, 2022.
- Younggyo Seo, Junsu Kim, Stephen James, Kimin Lee, Jinwoo Shin, and Pieter Abbeel. Multi-view masked world models for visual robotic manipulation. In *ICML*, 2023.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, YK Li, Y Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. Introduces GRPO.
- Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *ICML*, 2023.
- Ag2Manip team. Ag2manip: Learning novel manipulation skills with agent-agnostic visual and action representations. In *IROS*, 2024a.

ChatVLA team. Chatvla: Unified multimodal understanding and robot control with vision-language-action model. *arXiv preprint*, 2025a.

CLIP-DoRA team. Clip-dora: Weight-decomposed low-rank adaptation for efficient vision-language models. *arXiv preprint*, 2025b.

CoT-VLA team. Cot-vla: Visual chain-of-thought reasoning for vision-language-action models. *arXiv preprint*, 2025c.

Embodied CoT team. Robotic control via embodied chain-of-thought reasoning. *arXiv preprint arXiv:2407.08693*, 2024b.

FPC-VLA team. Fpc-vla: A vision-language-action framework with a supervisor for failure prediction and correction. *arXiv preprint*, 2026a.

GRAPE team. Grape: Generalizing robot policy via preference alignment. *arXiv preprint arXiv:2411.19309*, 2024c. Concurrent work on VLA preference alignment.

KVSharer team. Kvsharer: Efficient inference via layer-wise dissimilar kv cache sharing. *arXiv preprint arXiv:2410.18517*, 2024d.

MoFlow team. Moflow: One-step flow matching for human trajectory forecasting via implicit maximum likelihood estimation. *arXiv preprint*, 2025d.

ReBot team. Rebot: Scaling robot learning with real-to-sim-to-real robotic video synthesis. *arXiv preprint*, 2025e.

ReVLA team. Revla: Reverting visual domain limitation of robotic foundation models. *arXiv preprint arXiv:2409.15250*, 2024e.

Sentinel-VLA team. Sentinel-vla: A metacognitive vla model with active status monitoring for dynamic reasoning and error recovery. *arXiv preprint arXiv:2605.01191*, 2026b.

SnapFlow team. Snapflow: One-step action generation for flow-matching vlars via progressive self-distillation. *arXiv preprint arXiv:2604.05656*, 2026c. Validates our denoise-loop-targeting prescription (§5.4).

SpatialVLA team. Spatialvla: Exploring spatial representations for visual-language-action models. *arXiv preprint*, 2025f.

Survey team. A survey on vision-language-action models for embodied ai. *arXiv preprint arXiv:2405.14093*, 2026d.

VLA-ATTC team. Vla-attc: Adaptive test-time compute for vla models with relative action critic model. *arXiv preprint arXiv:2605.01194*, 2026e.

VLA Survey team. Vision-language-action models for robotics: A review towards real-world applications. *arXiv preprint arXiv:2510.07077*, 2025g.

Siyu Wang et al. Vla-cache: Towards efficient vision-language-action model inference via adaptive token caching. In *NeurIPS Workshop*, 2025. Targets autoregressive VLAs; we test transfer to flow-matching 0.5 (§4.5).

Jian Wen, Jian Zhang, et al. Tinyvla: Toward fast, data-efficient vision-language-action models for robotic manipulation. *arXiv preprint arXiv:2409.12514*, 2024.

Yuhui Xu, Lingxi Xie, Xiaotao Gu, Xin Chen, Heng Chang, Hengheng Zhang, Zhengsu Chen, Xiaopeng Zhang, and Qi Tian. Qa-lora: Quantization-aware low-rank adaptation of large language models. *arXiv preprint arXiv:2309.14717*, 2023.

Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. In *ICCV*, 2023.